

Gap analysis against Keka — overrides, loopholes, and what to build

A code-level audit of the CareVance payroll module set against Keka's documented payroll feature set, focused on the override surface, integrity loopholes, and employee master data.

Every claim below is anchored to a file and line in `D:\Caretime\backend`.

Audited 13 Aug 2026 · backend @ mtime 2026-08-13 · Keka sourced from help.keka.com (Aug 2026)

1. The headline
2. Scorecard vs Keka
3. The override gap — in depth
4. Loophole register — P0
5. Loophole register — P1 money
6. Statutory correctness gaps
7. Dead code register
8. Employee details gap
9. Override subsystem — design
10. Remediation roadmap
11. Where you already beat Keka
12. Verification notes

01 The headline

The gap between CareVance and Keka is **not mainly a missing-features gap**. You have more payroll code than the Keka comparison would suggest — 60+ payroll services, 21 filing generators, 36 PT jurisdictions, a real EPFO ECR writer, a correct gratuity helper, and a deliberately-correct refusal to fake Form 16 Part A. The gap is that a large share of that code is **not wired**, there is **no override layer at all**, there is **no audit trail on any change to an employee's pay**, and **"locked" does not lock anything**.

0

Authorization policies on ~60 payroll write endpoints

0

Rows ever written to `payroll_audit_logs`

3

Statutory overrides supported (Keka has 6 + component-level)

4

Independent income-tax engines that disagree

11

Payroll services that are never called

8

Payroll tables with no writer anywhere

Three findings that outrank everything else

CRITICAL 1. The override surface is empty — and the code says so

Keka's entire step-6 override screen plus its Payroll Admin → Operations override module (component, TDS, PF, ESI, PT, LWF, bulk Excel, mandatory comment) has **no counterpart**. Your own run checklist admits it:

`PayrollDepartmentController.php:2543-2548` — step `'overrides'`, status hardcoded `'no_action'`, detail *"Manual override tracking is not yet enabled"*. `:2455` — *"no `manual_override` flag yet"*.

A repo-wide grep for `override|manual_adjust|adhoc|one_time` returns zero payroll write endpoints.

CRITICAL 2. No audit trail exists on any pay figure

`PayrollAuditLog` and `payroll_audit_logs` exist. The only three references in the whole backend are the migration that creates it (`2026_03_25_120000...:195,205,211`) and a truncate list (`ClearMonthlyPayrollData.php:34`). **Zero writes, zero reads.**

Run-lifecycle events *are* audited via a raw insert (`writeRunAudit`, `PayrollDepartmentController.php:3028-3049`) — but it is never called by `processEmployeePayroll`, `quickSaveCtc`, `updateEmployeeTemplate`, `markItemPaid`, `processRunPayment`, `resetEmployeePayroll` or `approveArrear`. Every one of those moves money.

CRITICAL 3. A disbursed month is still editable

There is no DB constraint, trigger or model hook enforcing immutability — only nine independently-written `in_array()` guards. **Two of them test a status the system never writes.**

`EnhancedPayrollController.php:479` and `PayrollDepartmentController.php:1593` guard on `['paid', 'released']`. `'paid'` is never assigned as a run status (`PayrollMonthlyRun::CLOSED_STATUSES` uses `'disbursed'`). So `POST /payroll/arrears/{id}/approve` rewrites `gross_salary`, `pf_employee`, `esi_employee`, `pt`, `tds`, `net_pay` on a **locked, approved or already-disbursed** month — with no audit row.

02 Scorecard vs Keka

Each row is a capability Keka ships and an Indian payroll buyer will ask about in a demo.

DIMENSION	KEKA	CAREVANCE — VERIFIED	GAP
Salary structure	Range-based / custom / daily-wage structures per pay group; dynamic components with per-component flags; formula components; annual entry	Fixed columns duplicated across 3 tables. The run consumes exactly 3 fields : <code>basic_percentage</code> , <code>hra_percentage</code> , <code>conveyance_allowance</code> (<code>PayrollDepartmentController.php:1083-1095</code>); everything else is a residual. Dynamic <code>salary_template_components</code> has zero readers.	Severe
Component config	LOP-exempt, include-in-arrears, taxable, proof-required, FBP-eligible, payslip visibility, pay-outside-payroll, employee override allowed, max limit p.a.	8 of 9 absent. <code>is_taxable</code> exists as a column and is never read . Four more config columns (<code>impact</code> , <code>calculation_basis</code> , <code>is_compliance_component</code>) are written and never read.	Severe
Run workflow	6 explicit sequential steps, each with save & continue; enumerated pay actions; lock & confirm	Single <code>POST /payroll/process-and-pay</code> that walks draft→locked→approved→released atomically (<code>:4080</code>). A 6-step display exists but is read-only and gates nothing (<code>:2363-2575</code>).	Major
Overrides	Component, TDS (monthly + FY range), PF (EE/ER/VPF, formula-capable), ESI, PT, LWF, ad-hoc pay/deduct, bulk Excel, mandatory	None. Only <code>10P_days</code> and unstored <code>custom_deductions[]</code> . See §03.	Severe

DIMENSION	KEKA	CAREVANCE — VERIFIED	GAP
	comment, override reports		
Hold / stop payment	Hold processing vs hold payout as distinct semantics; void processing; void payout; already-paid; bulk import	Designed correctly (:3628-3631) — and inert . <code>is_payout_held</code> is missing from <code>PayrollItem::\$fillable</code> , so every write is silently dropped and held employees are paid.	Severe
Maker-checker	Configurable rules, criteria, AND/OR logic, approval chains, withdraw, activity log	A headcount heuristic: enabled only if the org has ≥3 admins (:2644-2654). Bypassable — <code>release</code> accepts a run in <code>locked</code> status (:3221), skipping approval entirely.	Major
Effective-dated comp	Pay Revisions with effective dates; salary timeline; bulk revision import	Accepting a revision does <code>\$template->update(['annual_ctc' => ...])</code> immediately; <code>effective_from</code> is stored and never read (<code>PayrollFilingController.php:2228-2256</code>). <code>CompensationTimeline</code> exists and its own docblock says it "does NOT yet feed the payroll engines".	Severe
Arrears / retro	Per-component arrear eligibility, arrear LOP, PF re-calc for arrear LOP, arrears report	Three implementations, three answers. The live one applies correctly but taxes a multi-month arrear with one month of incremental TDS (<code>EnhancedPayrollController.php:374-380</code>) and charges PF with no ₹15k ceiling (:361). No s.89(1)/Form 10E anywhere.	Major
Off-cycle payroll	First-class run type with its own payslips, overrides, lock & confirm	<code>OffCyclePayrollService</code> exists, has no route and no caller , and references an <code>Employee</code> model that does not exist.	Major
Statutory filings	PF ECR, ESIC ECR, 24Q FVU, PT, LWF; Form 16 via TRACES upload + signer app	21 generators. PF ECR is real. 24Q is FVU-shaped, not fileable (TDS amount written into the rate field, :494). 10 generators reference blade views	Moderate

DIMENSION	KEKA	CAREVANCE — VERIFIED	GAP
		that do not exist. Form 16 Part A correctly NOT faked.	
PT coverage	State + location-linked, slabs viewable per registration	36 jurisdictions with a correct Feb-extra rule for Maharashtra. Chandigarh slab breaches the Art.276(2) ₹2,500/yr cap; Punjab marked as levying none; TN/Kerala modelled monthly instead of half-yearly.	Moderate
F&F settlement	6 categories, per-item pay/void/already-paid, one-time or periodic partial, F&F statement	<code>unique(user_id)</code> → one F&F per employee, ever. Not wired to the exit flow. <code>EmployeeExit::settlement()</code> is a broken relation. Recovers only the first loan. No TDS, no s.10(10)/10(10AA).	Severe
FBP	Component flagged FBP-eligible, employee declares, claims with proof, unclaimed handling	Broken end to end. <code>FbpClaim::\$fillable</code> is the <i>abandoned</i> schema, so <code>POST /payroll/fbp/claims</code> cannot insert (NOT NULL <code>claimed_amount</code> dropped). FBP never reaches a payslip.	Severe
Loans	Categories, policies, bulk assign, foreclose, skip EMI, outside-payroll disbursal, 4 reports	Request/approve/reject/close only. No interest field at all , no schedule, no perquisite on concessional loan, no foreclosure or EMI skip. <code>closeLoan</code> writes no ledger row.	Major
Bank disbursal	NEFT/RTGS + bank-specific templates; explicit "we do not e-transfer"; gated payment automation with UTR	Batch + NEFT/RTGS CSV + real UTR discipline (good). Not idempotent — every bank-file download mints a new batch. A second path (<code>processRunPayment</code>) bypasses every exclusion check.	Severe
Reports	~60 named reports across 9 groups	3 endpoints: <code>payroll-register</code> , <code>statutory-register</code> , <code>bank-reconciliation</code> , plus a reconciliation report. No YTD, no IT computation, no HRA report, no investment/proof reports, no arrears/loan/F&F reports, no journal voucher.	Major

DIMENSION	KEKA	CAREVANCE — VERIFIED	GAP
GL / accounting	Journal voucher report; Zoho Books, QuickBooks, Tally, NetSuite	<code>GLMappingConfig</code> model exists; no export, no integration.	Major
Bonus Act	Not documented	Form C/D/E generators with correct 8.33–20%, ₹7,000 wage ceiling, ₹21,000 eligibility — but no bonus is ever paid ; the forms report a figure the engine never produces.	Moderate

03 The override gap — in depth

This is the specific thing you asked about, and it is the single widest gap. Keka's override surface is its strongest area: two separate override modules, six overridable quantities, formula-capable bulk import, a mandatory comment on TDS, and three dedicated override reports. CareVance has none of it.

OVERRIDE CAPABILITY	KEKA	CAREVANCE	EVIDENCE
Per-component earning override, one employee, mid-run	Yes	None	No endpoint writes <code>basic</code> / <code>hra</code> / <code>special_allowance</code> from input. <code>processEmployeePayroll</code> recomputes every component from <code>annual_ctc</code> and overwrites via <code>updateOrCreate</code> :1431-1498
Ad-hoc one-time payment	Yes	Proxy only	No dedicated endpoint. Abusable via <code>POST /payroll/arrears</code> , which accepts operator-stated <code>original_gross</code> / <code>revised_gross</code> with no cross-check against the actual item <code>EnhancedPayrollController.php:334-355</code> . Table <code>payroll_adjustments</code> has a <code>one_time_adjustment</code> kind and no controller .

OVERRIDE CAPABILITY	KEKA	CAREVANCE	EVIDENCE
Ad-hoc deduction	Yes	Partial	<code>custom_deductions[]</code> on the single-employee endpoint <code>:995-998</code> . Not a stored reviewable object — lives in <code>additional_components</code> JSON and is erased on any reprocess . Not forwarded by either bulk endpoint <code>:1713-1720</code>
TDS override — monthly	Yes	None	<code>tds</code> is only ever computed <code>:1165-1167</code> . Only an on/off flag exists.
TDS override — FY range, with mandatory comment	Yes	None	No range or FY-level override anywhere in the codebase.
PF override (EE share, ER share, VPF)	Yes	Template only	Template accepts the rates <code>:847-849</code> — but <code>PayrollAutoProcessService.php:559-563</code> hardcodes <code>min(\$basic,15000)</code> , <code>0.12</code> , <code>0.0833</code> , <code>0.0367</code> and ignores them. <code>vpf_employee</code> is fillable and never written by any code .
ESI override	Yes	Template only	Same split: honoured at <code>PayrollDepartmentController.php:1157</code> , hardcoded <code>0.0075 / 0.0325</code> at <code>PayrollAutoProcessService.php:583-584</code> .
Professional Tax override	Yes	None	Always <code>PTStateService::calculate()</code> <code>:1163</code> . Only <code>pt_enabled</code> + <code>pt_state</code> .
LWF override	Yes	None	Always <code>LwfCalculator::forMonth()</code> <code>:1268-1271</code>
LOP days override	Yes	Yes, crude	Accepted <code>:993</code> — but the bulk endpoints apply one flat value to every selected employee <code>:1674, :1711-1718</code> . No reason, no audit.
LOP reversal from a prior	Yes	None	<code>PayrollPeriodGuard.php:31</code> refuses back-dated attendance edits on closed periods and

Override Capability	KEKA	CareVance	Evidence
month			documents the remedy as "raise it as an arrear" — i.e. manual.
Bulk override via Excel/CSV	Yes	None	No import endpoint, no <code>maatwebsite/excel</code> . CSV appears only as output.
Reason / comment on an override	Mandatory on TDS	None	<code>createArrear</code> has <code>'reason' => 'nullable'</code> <code>:345</code> . The only mandatory reason in the module is on unlock <code>:2968-2974</code> .
Who-changed-what audit on an override	Partial (reports)	None	<code>payroll_audit_logs</code> has zero writes. <code>writeRunAudit</code> covers lifecycle only.
Maker-checker on an override	Comp change only	None	Arrears have approve/reject but no separation of duties — the same admin creates and approves <code>:334, :453</code> .
Dependent recalculation after override	Yes	Inconsistent	<code>10P_days</code> <i>does</i> cascade into PF/ESI/PT bases <code>:1117-1164</code> . An arrear does not — it adds operator-stated PF/ESI/TDS figures <code>:513-532</code> . <code>custom_deductions</code> are applied <i>after</i> all statutory computation <code>:1273</code> , so they never affect any base.
Override reports	TDS / ESI / PT	None	No override exists to report on.

Why this matters commercially

Every real payroll month has three or four employees who need something the engine cannot derive — a court-ordered deduction, a negotiated notice waiver, a TDS figure the employee's CA insists on, a statutory contribution frozen by a transfer. Keka handles all of these without a developer. In CareVance today the only routes to the same outcome are **editing the CTC** (which corrupts the annual figure and every downstream projection) or **a hand-crafted arrear** (which bypasses every guard and leaves no record). Both are worse than the problem.

04 Loophole register — P0, ship-blocking

These are exploitable or actively losing money today. Ordered by severity.

#	LOOPHOLE	WHAT ACTUALLY HAPPENS	LOCATION
L1	Any employee can read any colleague's CTC	<code>POST /payroll/leave-encashments</code> sits in the <i>self-service</i> route group, whose own header states "must resolve its subject from <code>\$request->user()</code> , never from a route parameter" — and then resolves it from <code>\$request->user_id</code> in the body. The 200 response returns <code>rate_per_day = annual_ctc / 12 / workingDays</code> . Any employee POSTs a 1-day encashment for a colleague and reads back their exact CTC. It also writes a <code>LeaveEncashment</code> row attributed to the victim.	<code>routes/.../payroll.php:35</code> <code>EnhancedPayrollControll</code>
L2	Payout hold is inert — held employees get paid	<code>is_payout_held</code> is not in <code>PayrollItem::\$fillable</code> . All three writers use mass assignment, so every write is silently dropped and the column is permanently <code>false</code> . Every reader — bank file, missing-bank-details, <code>cannotPay()</code> — is testing a constant. Separately, <code>releasePayout</code> guards <code>if (!\$item->is_payout_held) return 422</code> , so it returns 422 for every item, always: there is no working way to lift a hold. And nothing at disbursement time queries <code>StopPaymentFlag</code> at all.	<code>PayrollItem.php:13-106</code> <code>PayrollAutoProcessServi</code> <code>PayrollDepartmentContro</code> <code>PayrollDisbursementServ</code>
L3	A disbursed month can be rewritten	Two guards test <code>'paid'</code> , a run status the system never writes. Effective guard is <code>released</code> only. <code>approveArrear</code> therefore mutates <code>gross_salary</code> , <code>pf_employee</code> , <code>esi_employee</code> , <code>pt</code> , <code>tds</code> , <code>total_deductions</code> , <code>net_pay</code> and the run aggregates on a locked, approved or disbursed month, with no audit row. <code>quickSaveCtc</code> is editable on a disbursed month for the same reason.	<code>EnhancedPayrollControll</code> <code>PayrollDepartmentContro</code> <code>PayrollMonthlyRun.php:4</code>
L4	Zero authorization — a team lead is a payroll admin	No <code>app/Policies</code> directory. A repo-wide grep for <code>authorize(Gate:: ->can(abort_unless</code> in controllers and services returns nothing . The entire surface is one middleware line, <code>role:admin,manager</code> , and <code>EnsureUserHasRole</code>	<code>routes/.../payroll.php:98</code> <code>EnsureUserHasRole.php:4</code> <code>CheckPayrollPlan.php:11</code>

#	LOOPHOLE	WHAT ACTUALLY HAPPENS	LOCATION
		resolves <code>'manager'</code> to <i>any user with hierarchy level < 100</i> . So a team lead can lock/approve/release/disburse a run, download the NEFT CSV with every colleague's account number and IFSC, rewrite anyone's CTC, mark any item paid, and create+approve arbitrary arrears. Only 2 of ~60 methods do their own check.	
L5	Bank file is not idempotent — double-payment risk	<code>GET /payroll/runs/{runId}/bank-file</code> unconditionally calls <code>prepareBatch()</code> , which has no check for an existing pending batch and mints a fresh reference each time. Every download creates a new <code>BankTransferBatch</code> , new items, and a new file on disk. <code>disburseRun</code> then stamps items with <code>latest('id')</code> — so the reference written onto payslips may belong to a batch the operator never uploaded.	PayrollDepartmentContro PayrollDisbursementServ
L6	A second disburse path bypasses every exclusion	<code>POST /payroll/runs/{runId}/process-payment</code> accepts <code>approved</code> as well as <code>released</code> (skipping release), selects on <code>payment_status='pending'</code> with no <code>cannotPay()</code> and no hold check , marks negative-net and no-bank-account employees paid with invented references, and sets <code>status='disbursed'</code> with no <code>disbursed_at</code> , no <code>disbursed_by</code> , no audit, no batch reference and no payslip notification. The correct path does all of these.	PayrollDepartmentContro vs :4302, :4340-4399 routes/.../payroll.php:26
L7	Cross-tenant leak in the reconciliation report	<code>PayrollReconciliation</code> has no <code>BelongsToOrganization</code> trait <i>and</i> the table has no <code>organization_id</code> column — so <code>TenantIsolationTest</code> cannot catch it. <code>getReconciliationReport</code> queries on <code>month_year</code> only and returns <code>payrollItem.user.name</code> . Any admin of any tenant sees every other tenant's employee names and attendance deltas for that month.	PayrollReconciliation.p 2026_07_02_155250_...php PayrollAutoProcessContr
L8	<code>mark-paid</code> has no run-status check at all	Writes <code>payment_status='paid'</code> , <code>paid_at</code> and an invented <code>'PAY-' .md5(random_bytes(8))</code> reference on a <code>draft</code> run's item — directly contradicting the	PayrollDepartmentContro vs :4363-4377

#	LOOPHOLE	WHAT ACTUALLY HAPPENS	LOCATION
		codebase's own rule that only the bank's UTR is a valid reference.	
L9	Per-user attendance sync bypasses the period guard	The run-level sibling correctly calls <code>PayrollPeriodGuard::closedRunFor()</code> . The per-user version omits it, and rewrites <code>total_working_days</code> , <code>days_present</code> , <code>LOP_days</code> , <code>total_payable_days</code> on a locked/approved/disbursed item without recomputing money — the exact failure the guard exists to prevent.	<code>PayrollAutoProcessContr</code> vs :251, :262-267
L10	quick-process rewrites an approved month	No status guard at the controller. <code>autoApplyHolds()</code> <code>DELETE</code> s items regardless of run status; <code>calculateAllItems()</code> <code>update()</code> s every money column regardless of status, then forces <code>status='locked'</code> .	<code>PayrollAutoProcessContr</code> <code>PayrollAutoProcessServi</code>
L11	No concurrency control anywhere	A repo-wide grep for <code>lockForUpdate Cache::lock FOR UPDATE</code> across <code>app/</code> returns exactly one hit, in <code>PayslipController</code> . The 409 on background processing is a non-atomic read-then-write, so two simultaneous requests both dispatch. Only DB unique keys serialise — last writer wins on every money column.	<code>PayrollDepartmentContro</code> <code>ProcessPayrollRunEmploy</code>
L12	Maker-checker is a headcount heuristic and is bypassable	Enabled only when the org has ≥ 3 admins — a 2-admin org gets no separation of duties at all. Even when on, <code>releasePayrollRun</code> accepts a run in <code>locked</code> status, so approve can be skipped outright and the approve-time check with it.	<code>PayrollDepartmentContro</code>
L13	Unbounded numeric input on every money field	<code>annual_ctc</code> , <code>working_days</code> , <code>LOP_days</code> , <code>overtime_hours</code> and <code>custom_deductions.*.value</code> all have <code>min</code> and no max . A <code>percentage</code> -type custom deduction of <code>100000</code> yields a 1000× gross deduction. Overtime is <code>hours × rate × 2</code> with no ceiling. The arrear delta is unconstrained in sign <i>and</i> magnitude — a negative arrear is an unlimited ad-hoc deduction with no separate approval.	<code>PayrollDepartmentContro</code> <code>EnhancedPayrollControl1</code>

#	LOOPHOLE	WHAT ACTUALLY HAPPENS	LOCATION
L14	Silently-discarded writes (mass-assignment gaps)	Written by mass assignment, absent from <code>\$fillable</code> , so permanently zero in the DB: <code>days_absent</code> , <code>lop_deduction</code> , <code>total_lop_days</code> (masked by accessors, so model reads look right but every raw-SQL/BI report of loss-of-pay reads zero), and <code>payslips_notified_at</code> / <code>_status</code> / <code>_failed_count</code> (no accessor — simply lost).	PayrollItem.php:286, :2 PayrollMonthlyRun.php:4 PayrollDepartmentContro
L15	Loan balance drifts up on every reset	<code>resetEmployeePayroll</code> credits the loan balance back but does not delete the <code>PayrollLoanRecovery</code> ledger row or the <code>ReimbursementPayrollLink</code> . The next reprocess sees the recovery as already taken. Balance drifts up by one EMI per reset.	PayrollDepartmentContro

05 Loophole register — P1, wrong money on payslips

The root cause: four tax engines and three PF/ESI engines

The same employee's pay is computed by different code depending on which screen you look at. The engine wired to the *employee-facing tax simulator* is the worst of the four.

ENGINE	WHERE	WIRED TO	BEHAVIOUR
A	<code>PayrollCalculatorService.php:941/986</code>	the live payroll run	Best of the four. C + marginal relief, c
B	<code>SalaryCalculationService.php:329</code>	payslip preview	Flat 5% over ₹2.5 rebate. Loan EMI a <code>return 0</code> placeh
C	<code>PayrollFilingController.php:1536/1551</code>	<code>/payroll/tax-simulator/*</code> — <i>employee-facing</i>	FY 2024-25 slabs, Employee PF comp <code>min(basic, 1800</code> <code>\$pfAnnual</code> is unc

ENGINE	WHERE	WIRED TO	BEHAVIOUR
			<code>\$pfEmployee</code>). PT "tax_amount".
D	<code>TaxSimulatorService.php:14</code>	orphan	Double-counts ces already fixed in <code>EnhancedPayroll</code> this is the un-fixed

#	DEFECT	IMPACT	LOC
M1	HRA exemption is never applied	The producer emits key <code>section_hra</code> ; the consumer reads <code>hra_exemption</code> . Every old-regime employee's HRA claim silently evaluates to ₹0. Same for LTA. The sibling function does the opposite and adds HRA in full with no 10(13A) computation — two functions, two wrong answers.	Pay
M2	Only FY 2025-26 slabs exist	<code>TAX_SLABS_BY_FY</code> holds one year; anything else falls back to "latest known". You are in FY 2026-27 right now , so every TDS calculation is running on last year's table.	Pay
M3	TDS recomputed monthly, never trued up	<code>\$annualProjected = \$payableGross * 12</code> . A month with LOP annualises the reduced gross and under-deducts. No cumulative YTD reconciliation, so s.192 true-up never happens. A mid-year revision, bonus or regime switch never corrects earlier months.	Pay
M4	Previous-employer income not implemented	Grep for <code>previous_employer</code> returns nothing. No Form 12B. Every mid-year joiner is under-deducted for the whole year.	abs
M5	Reimbursements and perquisites are annualised ×12	Both are pushed into <code>custom_earnings</code> → <code>\$gross</code> → <code>× 12</code> . A one-off ₹20,000 reimbursement adds ₹2.4L of phantom annual income , and also inflates the ESI base and shifts the PT slab.	Pay
M6	Perquisites are paid out as cash	<code>autoSyncPerquisites</code> adds <code>monthly_value</code> to earnings → gross → net pay . A non-cash benefit is paid to the employee <i>and</i> reported on Form 12BA — double-counted.	Pay Pay

#	DEFECT	IMPACT	LOCATION
M7	PF deducted from leave encashment	A flat ₹1,800 is deducted regardless of encashment amount. Leave encashment is outside "basic wages" under EPF Act s.2(b) — nothing should be deducted.	Entire
M8	PF on arrears ignores the ₹15,000 ceiling	An employee already at the cap is charged a further 12% on the whole arrear.	Entire
M9	No EPS ₹1,250 cap, no eligibility rule	<code>eps = pfWages × 0.0833</code> gives ₹1,249.50 at the ceiling; EPFO's ECR edit check expects ₹1,250. No age-58 cut-off, no "joined after Sep 2014 above ₹15k ⇒ EPS-ineligible" rule.	Payment Payment
M10	EDLI wrong; employer cost understated	EDLI at <code>0.0017</code> (should be 0.5% of EDLI wages); admin charges 0.5% with no ₹500 monthly floor. The live engine computes neither at all — <code>totalEmployerContributions = pfEmployer + esiEmployer + gratuity</code> . The ECR challan will not reconcile.	Salary Payment
M11	ESI lock-in exists in only one engine	<code>EsiContributionPeriodService</code> is genuinely correct — Apr–Sep/Oct–Mar, year-boundary handled, mid-period crosser kept covered. But <code>SalaryCalculationService.php:155</code> does a bare ceiling test and drops the employee the month they cross, on earnings that wrongly include overtime and statutory bonus.	ESI vs
M12	Arrear taxed as one month	<code>tdsOnArrear</code> = (annual tax at revised – at original) / 12. A 6-month arrear deducts one month of incremental TDS. No employer-side statutory on arrears at all, so the PF ECR under-reports.	Entire
M13	No s.89(1) / Form 10E relief	Grep across <code>app/</code> and <code>routes/</code> returns nothing. Every arrear is taxed at the current year's marginal rate with no spread-back option.	absent
M14	Revision-letter arrear is 12× wrong	<code>(\$newCtc - \$oldCtc) * \$monthsElapsed</code> — an <i>annual</i> delta multiplied by a month count. It also writes <code>arrear_amount</code> and <code>rejection_reason</code> , neither of which exists as a column.	Payment
M15	Chapter VI-A gaps	80D hardcoded at ₹25,000 — no ₹50,000 senior band, no separate parents' limit (legal max ₹1,00,000). 80E and 80G uncapped in the calculator; 80G takes a flat 50%. 80CCD(2) employer NPS — deductible in the <i>new</i> regime — not modelled at all.	Payment
M16	Old-regime standard	Engine A uses ₹50,000 (correct); engine C applies ₹75,000 to the old regime (wrong).	Payment vs

#	DEFECT	IMPACT	LOC
	deduction inconsistent		
M17	Financial-year string format diverges	Canonical is <code>YYYY-YY</code> ; <code>EnhancedPayrollController::getCurrentFinancialYear()</code> returns <code>YYYY-YYYY</code> , so its declaration lookups and <code>bulkUpdateTaxRegime</code> never match a row .	Ent
M18	Reimbursement can be paid twice	<code>markPaid</code> sets <code>paid_at</code> / <code>payout_mode</code> without touching the payroll link, and <code>autoSyncReimbursements</code> filters on <code>status='approved'</code> + expense date — not on <code>paid_at</code> . A reimbursement marked paid <code>outside_payroll</code> is still picked up and paid again through payroll.	Rei Pay
M19	Two payroll engines disagree on overtime	Run engine: <code>gross / (workingDays × 8)</code> . Payslip engine: <code>basic / 208</code> with an unexplained magic number. The same employee's OT rate differs between screens.	Pay Sal
M20	Money is float end to end	No <code>bcmath</code> , no decimal library. Every money signature in <code>PayrollCalculatorService</code> is <code>float</code> . Two casts actively defeat the decimal column type: <code>SalaryComponent::default_value</code> and <code>SalaryTemplateComponent::value</code> cast <code>'float'</code> on <code>decimal(12,2)</code> columns — and those are precisely the dynamic-component tables. In <code>SalaryTemplate::calculateBreakdown</code> components are rounded individually <i>and</i> summed unrounded, so displayed line items need not add to the displayed gross.	Sal Sal Sal

06 Statutory correctness gaps

Professional Tax — 36 jurisdictions, several wrong

- **Chandigarh breaches the constitutional cap.** Slabs top out at ₹250/month = ₹3,000/yr, above the Article 276(2) ceiling of ₹2,500 per person per year. Chandigarh does not levy PT at all.

`PTStateService.php:265`

- **Nothing enforces the ₹2,500 annual cap.** `getAnnualLimit()` returns `maxMonthly × 12` = ₹3,600 for Maharashtra. `:539`

- **Punjab marked "No PT"** while the portal map simultaneously lists a Punjab PT portal. Punjab levies ₹200/month under the 2018 Development Tax Act. `:98-102 vs PayrollFilingService.php:798`
- **Half-yearly states modelled as monthly** — Tamil Nadu and Kerala are levied half-yearly by local bodies; no half-yearly return is produced. `:34-43, :124-136`
- **No gender dimension** — Maharashtra exempts women to ₹25,000/month; the code exempts everyone only to ₹7,500. `:20`
- Arunachal Pradesh missing from a list labelled "States (28)"; J&K classified as a state; DNH and Daman & Diu listed separately. `:302, :311, :334-335`
- **Correct:** the Maharashtra February extra instalment is handled properly — applied only when the employee is in the top band, so a ₹9,000 earner keeps ₹175. `:410-417, :467`
- The employee-facing simulator invents a separate annual PT table peaking at **₹6,000/yr** — 2.4× the constitutional maximum. `PayrollFilingController.php:1570-1588`

LWF — one shared config, mostly wrong frequency

- Deduction and return read the same table — a genuine improvement over the previous divergence. Unconfigured states yield ₹0 rather than a guessed default. `LwfCalculator.php:37-42`
- **14 of 15 states are set to monthly**; most LWF Acts are half-yearly or annual. Gujarat, the one `bi_annual` state, is set to months `[1,7]` — Gujarat's half-years end June and December. `PayrollFilingService.php:904-928`
- **Rajasthan listed as having no LWF Act** — it has one (1954). `:924-927`
- **Employer-side LWF is never computed** — the table stores only the employee amount and `SalaryCalculationService.php:203` sets employer = employee. Most states set employer at 2–3×.
- **The LWF return ignores the deduction it reports** — it prints a flat per-employee amount from the config rather than reading `payroll_items.lwf`, so a mid-period joiner is still billed. `:981-988`

Filings — 21 generators, 10 of them cannot run

GENERATOR	OUTPUT	STATUS
PF ECR	<code>.txt</code> , 11-col <code> </code>	Real EPFO ECR — validates properly; declared wages reconciled to the same LOP-reduced base
ESI challan	<code>.xls</code> , 6-col	Portal template — but hardcodes <code>filing_ready: true</code> with no IP-number validation;

GENERATOR	OUTPUT	STATUS
		defaults every LWD to month-end and reason code to <code>'0'</code>
Form 24Q	<code>.txt</code> , <code>^</code> - delimited	FVU-shaped, not fileable — the DD record puts the TDS amount in the TDS-rate field ; no Annexure II, no real challan BSR/CIN (deposit date fabricated as <code>date('dmY')</code> , serial hardcoded <code>'001'</code>)
Form 16 Part B / 12BA	PDF	Correct — and Part A is deliberately <i>not</i> generated locally, with a TRACES upload path instead. The single most correct decision in the module.
PT / LWF / Bonus C,D,E	<code>.txt</code>	Reference only , honestly flagged as such in code
Form 1, 2, 6, 19, 31, 124, eShram, SE reg, Shram card, UAN activation	PDF	10 generators, blade views do not exist . <code>generateForm19</code> additionally has a fatal bug — <code>->count()</code> called on a PHP array at <code>:1438</code> — and queries a <code>payroll_run_id</code> on settlements that is never populated, so it would be empty even once the view exists.

Two systemic issues:

- **The pre-flight validator reads settings keys the setup wizard never writes.** It reads `settings.statutory.establishmentCode`, `settings.esi_code`, `settings.tan_number`, `settings.pan_number`; the writer uses `settings.payroll.statutory.{establishmentCode,esiCode,tan,pan}`. **A fully-configured org fails every check.** It also bypasses `User::statutoryId()` and reads the profile column directly, so anyone whose IDs live in `employee_government_ids` is reported as missing.
`PayrollFilingValidatorService.php:68,82,105,116,133,141,177`
- **The entire filing review workflow is unreachable.** `submitForReview`, `approveFiling`, `rejectFiling`, `reviewQueue` have **no routes** — along with the migration written to support them. `PayrollFilingController.php:636, :670, :692, :772`
- **Correct:** `generateAllFilings` is well-built — per-generator try/catch, `InvalidArgumentException` treated as skip, everything else recorded as a named failure. The ten missing views cost ten failure entries, not the batch. `:1952-2038`

Full & Final — structurally blocked

- `unique(user_id)` **means one F&F per employee, forever.** The code tells the operator to "reject the existing one first" — doing so leaves the rejected row occupying the index, so the replacement `create()` throws and surfaces as a 500. It also blocks rehires.
`2026_06_10_120003...:83; EnhancedPayrollController.php:615-626, :706`
- **Not wired to the exit flow.** `ExitService` creates the exit, checklist and asset-return items and never touches `FullAndFinalSettlement`. `advance($exit, 'settlement')` only flips a stage string.
- `EmployeeExit::settlement()` **is a broken relation** — it resolves to foreign key `employee_exit_id`, a column that does not exist. Loading it raises "column does not exist".
`EmployeeExit.php:83`
- Recovers **only the first loan** (`->first()`). Asset recovery, pending reimbursements, salary arrears and comp-off value have columns and are **never populated**. `:669-673`
- **No TDS on settlement at all** — no s.10(10) gratuity exemption, no s.10(10AA) leave-encashment exemption, no s.192(5).
- Leave encashment and current-month salary are computed off **CTC**, not gross — overstating both by the employer PF and gratuity share. `:653-667`
- `FullAndFinalSettlement::calculateGratuity()` is a duplicate with **neither the 5-year floor nor the ₹20L cap**, sitting one autocomplete away from the correct helper. A death-in-service under 5 years currently pays ₹0. `FullAndFinalSettlement.php:144-147`

FBP — broken end to end

Two conflicting `fbp_claims` schemas shipped; `FbpClaim::$fillable` is the **abandoned** one. Consequences chain:

- `POST /payroll/fbp/claims` inserts `amount` + `claim_date` and omits `claimed_amount` (NOT NULL) and `fbp_allocation_id` (NOT NULL FK) — **the employee-facing claim endpoint cannot succeed**. `PayrollFilingController.php:1720-1728`
- Approve/reject write `reviewer_id` / `reviewed_at` — nonexistent columns — and never set `approved_amount` OR `month_year`.
- `autoSyncFbp` filters on `month_year` and sums `approved_amount`, both permanently null. **FBP never reaches a payslip.**
- `FbpService` queries `financial_year_id`; the column is `financial_year`. Its own writer uses the correct name — reader and writer disagree.
- No annual-limit enforcement, no carry-forward, no unclaimed handling, no taxable-on-lapse. The tax logic itself is sound — it just runs on rows that are never populated.

Perquisites — the API writes a table payroll never reads

- `perquisite_records` is read by the payroll engine and Form 12BA — and has **no write endpoint**.
- `employee_perquisites` is what the only two prerequisite routes write — and **nothing else reads it**.
- A prerequisite created through the API is invisible to payroll, to TDS and to Form 12BA.
- `PerquisiteCalculator` is an orphan. Its accommodation rates (15%/10%) are the **pre-01.09.2023** rates; current Rule 3 is 10%/7.5%/5% by population band. No chauffeur addition. Concessional loan (Rule 3(7)(i)), credit cards, club fees, medical, education, movable-asset transfer and gifts are all absent.

07 Dead code register

This is the highest-leverage section. A large fraction of what looks like Keka parity is already written and simply not connected — but several pieces would **fatal on first call** if you wired them naively, because they reference models and tables that do not exist. Treat this as a decision list: wire, fix, or delete.

Services that are never called

SERVICE	RUNNABLE?	WHY, AND WHAT TO DO
<code>OffCyclePayrollService</code>	No	Uses <code>Employee::findOrFail()</code> . There is no <code>Employee</code> model — <code>SalaryCalculationService.php:26</code> states flatly "Employees are users". Rewrite against <code>User</code>; this is a genuine Keka parity feature.
<code>OnDemandSalaryService</code>	No	<code>DB::table('ewa_transactions')</code> with no <code>organization_id</code> filter and no such column — cross-tenant by construction. Delete or rebuild.
<code>GarnishmentService</code>	No	<code>Employee::with('user')</code> . Court-ordered deductions are a real requirement — fold into the new override subsystem instead.
<code>ProductivityPayrollService</code>	No	Writes <code>kind => 'productivity_bonus'</code> against an enum that does not include it → constraint violation. Also passes <code>'reason'</code> , not in <code>\$fillable</code> .

SERVICE	RUNNABLE?	WHY, AND WHAT TO DO
MultiPayGroupService	No	Imports <code>PayCalendar</code> , <code>PayrollRun</code> , <code>Employee</code> — none exist. Writes <code>cycle_type</code> , <code>cutoff_day</code> , <code>payment_day</code> to <code>pay_groups</code> , none of which are columns; writes <code>pay_group_members</code> , real table is <code>pay_group_assignments</code> .
CtcPlannerService	No	Imports <code>PayrollRun</code> , <code>Department</code> , <code>Employee</code> ; writes <code>DB::table('salary_revisions')</code> — no migration creates that table.
CostCentreService	No	Imports <code>CostCentre</code> , <code>Employee</code> ; queries <code>cost_centre_allocations</code> and <code>cost_centres</code> — neither exists; reads <code>\$user->cost_centre_id</code> , not a column.
DepartmentBudgetService	No	Queries <code>department_budgets</code> (absent) and <code>users.department_id</code> (departments are <code>groups</code> + <code>group_user</code>). Sums <code>net_pay</code> off the retired <code>payrolls</code> table.
VariablePayEngine	Yes	Orphan. <code>PayrollFilingController::calculateVariablePay</code> reimplements a <i>subset</i> — fixed and percentage only. Slab and performance-based variable pay exist only in the orphan. Wire the engine, delete the reimplementation.
TaxOptimisedStructureService	Yes	Runnable, never called. Cheap win for a CTC-planner screen.
CompensationTimeline	Yes	Deliberate orphan — its own docblock says it "does NOT yet feed the payroll engines". This is the missing piece of effective-dated compensation. Wire it.
SalaryRevisionService	Yes	Superseded by a worse reimplementation in <code>PayrollFilingController</code> that drops the breakdown snapshots, the PDF, and the deliberate "default to first of <i>next</i> month" logic. Restore the service, retire the controller version.
ArrearCalculatorService	Partly	Orphan and worse than the live path : <code>total_arrear = basic + pf_employee + esi_employee</code> — it <i>adds</i> the employee's own deductions to the payout.

SERVICE	RUNNABLE?	WHY, AND WHAT TO DO
		Also reads a <code>total_earnings</code> key the calculator never returns. Delete.
<code>PerquisiteCalculator</code>	Partly	Only writer of the table payroll actually reads. Rates are stale (pre-Sep-2023). Fix rates, then wire as the sole perquisite writer.
<code>TaxSavingsRecommender</code>	No	Reads <code>section_80c_total</code> etc. — columns absent from <code>employee_tax_declarations</code> ; references <code>App\Models\Employee</code> . The route that calls it (<code>/payroll/tax-savings/recommendation</code>) is live and returns HTTP 200 with every figure zero .
<code>TaxSimulatorService</code>	Yes, wrong	Double cess. The same bug was fixed elsewhere; this is the un-fixed twin. Delete.
<code>PayrollApprovalService</code>	Yes	Implements a multi-level approval workflow that is entirely unreachable. Wire it in place of the headcount heuristic.

Tables with no writer

`salary_slab_components` · `salary_lookup_components` · `ctc_range_bands` · `daily_wage_structures` · `employee_salary_assignments` · `payroll_structures` · `salary_template_components` · `payroll_profiles` · `payroll_audit_logs`

Two of these have *readers* that will fail: `ctc_range_bands` is queried as `ctc_bands` by both its endpoints (undefined-table error), and `employee_salary_assignments` is read by `PayrollReadinessService.php:159-163`, which therefore always reports nobody assigned. `payroll_structures` even carries a correct `scopeEffectiveOn()` with zero callers — effective dating was designed and abandoned twice.

Code that writes to things that do not exist

WRITTEN	BY	REALITY
<code>salary_revisions</code> table	<code>CtcPlannerService.php:35</code>	no migration

WRITTEN	BY	REALITY
cost_centres , cost_centre_allocations	CostCentreService.php:31,39	no migration
department_budgets	DepartmentBudgetService.php:19,32	no migration
pay_group_members	MultiPayGroupService.php:50	real table is pay_group_
salary_revision_letters.arrear_amount , .rejection_reason	PayrollFilingController.php:2191, :2280	not in any m
pay_groups.cycle_type , .cutoff_day , .payment_day , .week_start_day , .approval_workflow_id	MultiPayGroupService.php:33-37	not columns, \$fillable

Columns written and never read

employee_payroll_templates : cca_amount , education_allowance , hostel_allowance , internet_allowance , transport_allowance , uniform_allowance , books_periodicals , fuel_maintenance , nps_employee_percentage , component_settings . Your HR team can fill in a Books & Periodicals allowance and it will never appear on a payslip.

salary_components : impact , calculation_basis , is_taxable , is_compliance_component , default_value , meta , value_type .

pay_groups : pay_frequency , pay_day , pay_day_type , salary_template_id .

salary_formulas : effective_from , effective_to — stored, and the evaluator filters only on is_active .

The formula engine cannot save a usable formula

SalaryComponentController::saveFormula validates with new SalaryFormulaEngine() — a **fresh engine with no variables set**. replaceVariables() substitutes nothing, and evaluateSimple() was deliberately hardened to fail loudly on leftover alphabetic text. So:

```
CTC * 0.08           → INVALID
MAX(Basic, 15000)    → INVALID
Basic * 0.1          → INVALID
```

```
50000                                → VALID
IF(CTC > 500000, 50000, 0)           → VALID ← by accident
```

The `IF()` cases pass only because the unsubstituted `"CTC "` casts to `0`, `0 > 500000` is false, and the false branch is returned without the variable ever being parsed. **Validation depends on which branch a variable-free comparison happens to select.** With variables set the engine works correctly — the save gate is the only thing broken. Net effect: `salary_formulas` can hold constants only. `SalaryComponentController.php:200-206; SalaryFormulaEngine.php:11, :102-118, :190-206, :237-239`

There are also **three independent expression evaluators** in the codebase — `SalaryFormulaEngine`, the inline percentage handling in `SalaryTemplate::calculateBreakdown`, and `PayrollFilingController::{safeEval,tokenize,parseExpression}` behind `/payroll/formula-engine/{evaluate,validate}`. The "validate" endpoint the UI would call and the validator that actually gates saving **share no code and disagree.**

Live endpoints that cannot succeed

- `POST /payroll/fbp/claims` — NOT NULL violation (see §06)
- `POST /payroll/fbp/claims/{id}/approve|reject` — write nonexistent columns
- `POST /payroll/perquisites` — writes a table nothing reads
- `GET /payroll/compensation/ctc-bands`, `POST /payroll/compensation/find-ctc-band` — query `ctc_bands`, table is `ctc_range_bands`
- `GET /payroll/compensation/daily-wage-structures` — no writer exists, always returns `[]`
- `GET /payroll/tax-savings/recommendation` — HTTP 200, every figure zero
- `GET /payroll/auto/checklist-status/{runId}` — **guaranteed 500**: ignores the route param, and calls `$this->checklist->getRunStatus()`, a method `PayrollChecklistService` does not have
- `POST /payroll/filings/generate/form-19` — fatal `->count()` on an array
- 10 declaration-form generators — missing blade views

08 Employee details gap

What Keka requires and validates before payroll will run

Keka's employee statutory block is explicit and bulk-editable: **Employer PF Establishment ID, EPF/EPS contribution type, UAN, PF Number, PF Joining Date, Name on PF Account, Employer ESI Establishment ID, Employee ESI Number, PT Establishment ID, state and registered**

location (auto-derived), LWF eligibility, plus explicit "not available / not eligible" checkboxes. PAN has its own management article and is both the Form 16 match key and the payslip password. Errors surface as *Error Name / Error Details / Ways to Fix*, with an **Unprocessed Employees** list on the run itself.

FIELD	STORED	VALIDATED	BLOCKS A RUN?	NOTE
PAN	Yes	On demand	No	Lives in two places — profile column or <code>employee_government_ids</code> . <code>User::statutoryId()</code> resolves both; the filing validator bypasses it and reads the profile column directly, so anyone using the other store is reported missing.
UAN / PF number	Yes	On demand	No	Same dual-store problem. PF ECR does validate at generation time — but that is after the money has moved.
PF joining date, name-on-PF	No	—	—	Not modelled. Required by EPFO for a clean ECR.
ESI / IP number	Yes	On demand	No	ESI challan hardcodes <code>filing_ready: true</code> with no IP-number validation at all .
Establishment IDs (PF / ESI / PT / LWF)	Org + pay-group	No	No	<code>PayGroupFilingDetail</code> exists and is CRUD-only : <code>getFilingDetailForState()</code> has zero callers and no filing generator references it. Statutory registration is effectively org-level only.
Bank a/c + IFSC	Yes	At file time	Partly	Excluded from the batch by <code>cannotPay()</code> — but the second disburse path skips the check entirely (L6).
PT state	Yes	No	No	<code>department_payroll_templates.pt_state</code> defaults to <code>'maharashtra'</code> — directly contradicting the house rule that an unset state must yield ₹0. A department template seeded with defaults stamps Maharashtra onto every new joiner in it.

FIELD	STORED	VALIDATED	BLOCKS A RUN?	NOTE
				<code>PayrollCalculatorService::calculateP</code> carries the same default as a trap for the next caller.
LWF eligibility	Flag only	No	No	No state-level eligibility, no location link.
Tax regime	Yes	No	No	Bulk update exists but uses the wrong FY string format, so it matches nothing.
Cost centre	No	—	—	The service references <code>users.cost_centre_id</code> ; it is not a column. No cost-centre reporting is possible.
Pay group	Yes	—	No	Decorative at runtime — see below.

Nothing hard-blocks a run

`PayrollValidationService`, `PayrollChecklistService` and `PayrollReadinessService` all exist and all are **advisory**. `lockPayrollRun` never calls `validatePayrollRun()` — validation is reachable only through a separate opt-in endpoint. And `PayrollReadinessService`'s salary-assignment check reads a table that has no writer, so it always reports nobody assigned. The only true hard requirement is `annual_ctc > 0`.

Pay group is decorative

- `payroll_monthly_runs` has **no** `pay_group_id` and is uniquely keyed `(organization_id, month_year)` — every pay group in an org lands in the same monthly run.
- `pay_frequency`, `pay_day`, `pay_day_type` are validated, echoed in JSON, and **read by nothing**. No code derives a period, a cutoff or a pay date from them.
- `pay_groups.salary_template_id` is never used to resolve a structure at calculation time.
- What it *does* do: copy `statutory_rules` JSON into each employee's template at three points. Once copied, editing an employee's template diverges from the group silently.

Employee self-service — what exists vs what Keka gives

ESS CAPABILITY	CAREVANCE	NOTE
Payslip list & download	Yes	GET /payroll/my/payslips + PDF download
Tax declaration (view / save / submit)	Yes	/payroll/my/declaration , /items , /submit
Proof upload + Form 12BB	Yes	/payroll/tax-proofs , /my-12bb/{fy} — a genuine strength
Reimbursement claim	Yes	/payroll/reimbursements + /mine
Loan request & my-loans	Yes	No interest, no schedule visible
Payslip YTD	Partial	/payroll/payslips/{id}/ytd exists; PayslipYtdHistory maintenance not verified
Salary breakup / structure view	Admin-side	SalaryBreakdownService is display-only and wired to the admin employee card, not to ESS
Tax simulator / regime compare	Wrong numbers	Engine C — FY24-25 slabs, no 87A, PF = basic (\$05)
Salary timeline / revision history	Missing	No effective-dated history to show
IT computation statement	Missing	No endpoint
Form 16 download	Missing in ESS	Upload path exists admin-side; no employee download route
FBP declaration	Broken	Claim endpoint cannot insert (\$06)
Salary / employment certificate	Missing	Common India ask (visa, loan)
Bank detail self-update	Missing	—
Leave encashment request	Data leak	See L1 — resolves the subject from the request body

09 Override subsystem — proposed design

This is the one build that closes the widest gap and simultaneously fixes L3, L13 and the missing audit trail. Design it once, properly, rather than adding override fields to existing endpoints.

Principles

1. **An override is a first-class row, not a mutated payroll item.** The engine keeps recomputing from source; the override is applied as a documented, reversible layer on top. This is what makes a payslip explainable and an override reportable.
2. **Every override carries who, when, why.** Reason mandatory on statutory overrides — matching Keka's mandatory comment on TDS, and going one better by applying it to all six.
3. **Overrides are scoped to a run and a lifecycle state.** Allowed while `draft` / `processing`; require unlock after `locked`; forbidden after `disbursed` — enforced in *one* place, not nine.
4. **Declare the recalculation contract.** Each override type states whether dependents recompute. A component override recomputes PF/ESI/PT/TDS; a TDS override does not.

Schema

`payroll_overrides`

```
id, organization_id, payroll_run_id, user_id, payroll_item_id
scope          -- 'component' | 'statutory' | 'adhoc' | 'lop' | 'hold'
target         -- 'basic' | 'hra' | ... or 'tds' | 'pf_employee' | 'pf_employer' | 'vpf'
               --                  | 'esi_employee' | 'esi_employer' | 'pt' | 'lwf'
mode          -- 'absolute' | 'delta' | 'formula'
value          decimal(14,2)
computed_value decimal(14,2) -- what the engine would have produced
effective_from date, effective_to date -- Keka's FY-range TDS override
reason         text          NOT NULL for scope='statutory'
source         -- 'ui' | 'import' | 'api' | 'fnf'
import_batch_id
status         -- 'pending' | 'approved' | 'rejected' | 'applied' | 'reversed'
created_by, approved_by, approved_at, applied_at, reversed_at, reversal_reason
UNIQUE (payroll_run_id, user_id, target, effective_from)
```

`payroll_overrides_audit` -- append-only; no UPDATE, no DELETE

```
id, payroll_override_id, action, actor_id, actor_role,
before_json, after_json, ip, user_agent, created_at
```

Application order in the engine

1. resolve compensation as of the period ← CompensationTimeline (wire it)
2. apply LOP / attendance
3. apply scope='component' overrides ← recompute dependents from here down
4. compute PF / ESI / PT / LWF
5. apply scope='statutory' overrides ← terminal, no cascade
6. compute TDS on YTD basis
7. apply TDS override if in effective range ← terminal
8. apply scope='adhoc' payments and deductions
9. apply holds → is_payout_held / item suppression
10. persist item + snapshot of every override applied

Endpoints to add

ENDPOINT	PURPOSE
POST /payroll/runs/{run}/overrides	Create one override. Validates target, bounds, run state, reason.
POST /payroll/runs/{run}/overrides/import	Excel/CSV bulk — Keka parity. Returns a per-row error report.
POST /payroll/overrides/{id}/approve reject	Maker-checker. Creator ≠ approver, enforced.
DELETE /payroll/overrides/{id}	Reversal — writes a <code>reversed</code> row, never deletes.
GET /payroll/runs/{run}/overrides	The override register. Becomes the TDS/ESI/PT override reports Keka ships.
GET /payroll/overrides/audit?user=&from=&to=	The who-changed-what trail that does not exist today.

Bounds to enforce at the request layer (closes L13)

annual_ctc	max 100_000_000	
working_days	1..31	
lop_days	0..days_in_month	
overtime_hours	0..(days_in_month * 12)	
component value	0..(3 * computed_value)	— beyond this, second approver
statutory value	0..(2 * computed_value)	
adhoc deduction	0..net_pay_before_adhoc	

10 Remediation roadmap

Sequenced so that each phase is independently shippable and each one makes the next safer. Phase 0 is not optional — several of its items are losing money or leaking data today.

Phase 0 — Stop the bleeding 3–5 days • no schema change

REF	FIX	CHANGE
L1	Close the CTC leak	Resolve the subject from <code>\$request->user()</code> in <code>requestLeaveEncashment</code> ; drop <code>user_id</code> from the request. Add a test that fails if any route in the self-service group reads a subject from input.
L2	Make payout hold work	Add <code>is_payout_held</code> to <code>PayrollItem::\$fillable</code> . Backfill from <code>StopPaymentFlag</code> . Add a <code>StopPaymentFlag</code> check to <code>prepareBatch</code> and <code>buildBankFilePayload</code> as defence in depth.
L3	Kill the dead status guards	Replace every <code>in_array(\$run->status, ['paid','released'])</code> with <code>\$run->isClosed()</code> using <code>PayrollMonthlyRun::CLOSED_STATUSES</code> . Grep for <code>'paid'</code> as a run status and remove it everywhere.
L4	Split the payroll role	Introduce <code>payroll_admin</code> . Gate lock / approve / release / disburse / bank-file / CTC-edit / arrear-approve on it, not on <code>role:admin,manager</code> . Interim: change the middleware to <code>role:admin</code> only.
L5	Make the bank file idempotent	<code>prepareBatch</code> returns the existing pending batch for the run instead of minting a new one. Add <code>UNIQUE(payroll_run_id) WHERE status='pending'</code> .
L6	Delete the second disburse path	Remove <code>processRunPayment</code> and its route; point the UI at <code>disburseRun</code> .
L7	Scope the reconciliation table	Add <code>organization_id</code> to <code>payroll_reconciliation</code> , add <code>BelongsToOrganization</code> , backfill from the run.

REF	FIX	CHANGE
L8, L9, L10	One immutability guard	Add a <code>saving</code> observer on <code>PayrollItem</code> that refuses any write when the parent run is closed. Delete the nine ad-hoc guards. This is what makes L8/L9/L10 unreachable rather than individually patched.
L14	Fix the mass-assignment gaps	Add <code>days_absent</code> , <code>lop_deduction</code> , <code>total_lop_days</code> to <code>\$fillable</code> ; add <code>payslips_notified_*</code> to <code>PayrollMonthlyRun::\$fillable</code> . Backfill the three LOP columns from the accessors.
M2	Add FY 2026-27 slabs	Extend <code>TAX_SLABS_BY_FY</code> . Then make <code>slabsFor()</code> throw on an unknown FY instead of silently falling back.
M1	Fix the HRA key mismatch	One-line change: emit <code>hra_exemption / lta_exemption</code> from <code>exemptionKey()</code> . Add a regression test asserting a declared HRA reduces old-regime tax.
—	Default PT state	Drop the <code>'maharashtra'</code> default on <code>department_payroll_templates.pt_state</code> and the two defaults in <code>PayrollCalculatorService</code> .

Phase 1 — The override & audit layer 3–4 weeks • the Keka-parity item

1. Build `payroll_overrides` + `payroll_overrides_audit` per §09.
2. Refactor the engine to the 10-step application order. This forces the single calculation path that Phase 2 depends on.
3. Ship component, TDS (monthly + FY-range), PF (EE/ER/VPF), ESI, PT, LWF, ad-hoc payment, ad-hoc deduction.
4. Excel/CSV bulk import with a per-row error report. Add `maatwebsite/excel`.
5. Wire `PayrollApprovalService` as the maker-checker instead of the ≥ 3 -admins heuristic. Creator $\neq$ approver, enforced in code.
6. Override register + audit endpoints → these become the TDS/ESI/PT override reports.
7. Retire the arrear-as-override workaround: `createArrear` validates against the actual payroll item.

8. Fold garnishment in as `scope='adhoc', target='garnishment'` rather than reviving the broken service.

Phase 2 — One engine, one truth 3-4 weeks

1. **Delete engines B, C and D.** Point `PayslipController` and the tax simulator at `PayrollCalculatorService`. This alone fixes M6 (std deduction), the FY24-25 slabs, PF-as-basic, the flat-5% payslip TDS, the double cess, and the overtime divisor divergence.
2. Hardcoded statutory rates out of `PayrollAutoProcessService` — read the template, as the other engine does (fixes the PF/ESI override half-gap).
3. Move reimbursements and perquisites **out of** `custom_earnings` into their own non-annualised, non-ESI-bearing buckets (M5). Perquisites must add to the tax base without adding to net pay (M6).
4. Introduce **YTD-based TDS**: tax on YTD actual + projected remaining, minus TDS already deducted, spread over remaining months (M3). Add previous-employer income capture (M4).
5. Add EDLI and admin charges to the live engine with the correct rates and the ₹500 floor; add the EPS ₹1,250 cap and eligibility rule (M9, M10).
6. Money to `decimal` discipline: remove the two `'float'` casts, round once at the boundary (M20).

Phase 3 — Effective-dated compensation 3-4 weeks

1. Make `salary_revision_letters` the compensation timeline, or promote it to a proper `compensation_records` table. Either way: **a revision creates a dated record; it never overwrites `annual_ctc`**.
2. **Wire** `CompensationTimeline` — it already segments a month into rate periods. That single change makes mid-month revisions correct.
3. Fix the 12× arrear formula and add the two missing columns (M14).
4. Back-dated revision → automatic arrear against the true old rate, using `template_snapshot` for closed months. Fix the stale `'paid'` status list in the arrear detector.
5. Multi-month arrear TDS spread (M12), then **s.89(1) + Form 10E** (M13).
6. Expose a salary timeline in ESS.

Phase 4 — Statutory correctness 4 weeks

1. PT: remove Chandigarh, restore Punjab, enforce the ₹2,500 annual cap in `getAnnualLimit()`, add the gender dimension, model TN/Kerala half-yearly. Re-verify Karnataka, Gujarat, Odisha, HP against current notifications.

2. LWF: correct the frequency for all 15 states, fix Gujarat's half-year months, add Rajasthan, add employer-side amounts, and make the return read `payroll_items.lwf`.
3. Fix the validator's settings-key mismatch — this is why pre-flight is red on a correctly-configured org — and route it through `User::statutoryId()`.
4. Write the 10 missing blade views, or delete those generators. Fix the `generateForm19` fatal.
5. Form 24Q: put TDS rate in the rate field, add Annexure II, capture real challan BSR/CIN. Or reposition it honestly as "24Q source data for your RPU".
6. Remove the hardcoded `filing_ready: true` from ESI challan, full ECR and the ten declaration forms.
7. Wire the filing review workflow — controller methods and migration already exist, they just have no routes.

Phase 5 — Feature parity `rolling`

- **F&F rebuild:** drop `unique(user_id)` → `unique(user_id, exit_id)`; wire to `ExitService`; fix the `EmployeeExit::settlement()` relation; recover all loans; populate asset recovery and pending reimbursements; add TDS with s.10(10) and s.10(10AA); support periodic partial settlement; delete the unguarded duplicate `calculateGratuity()`.
- **FBP:** align `FbpClaim::$fillable` with the live schema, fix `financial_year_id` → `financial_year`, enforce annual limits, add unclaimed handling and taxable-on-lapse.
- **Perquisites:** collapse to one table, wire `PerquisiteCalculator` as its only writer, update accommodation rates to post-Sep-2023 Rule 3, add chauffeur, add Rule 3(7)(i) concessional loan.
- **Loans:** add interest rate + amortisation schedule, foreclosure, EMI skip, moratorium; make `closeLoan` write a ledger row; fix the reset drift (L15).
- **Off-cycle payroll:** rewrite `OffCyclePayrollService` against `User` and give it a route. Real Keka parity item.
- **Pay group as a real dimension:** add `pay_group_id` to `payroll_monthly_runs`, key on `(org, pay_group, month)`, and make `pay_frequency / pay_day` actually drive the period.
- **Component model:** make `salary_template_components` the real structure and add the eight missing per-component flags — LOP-exempt and include-in-arrears first, since those two change payslip figures.
- **Formula engine:** fix the save gate to validate against a variable-populated engine; delete the two rival evaluators; honour `effective_from / effective_to`.
- **Reports:** YTD family, IT computation, HRA, investment declaration & proof submission, arrears, loans, F&F, one-time payments/deductions, journal voucher. Wire `GLMappingConfig` and ship a Tally/Zoho-compatible JV export — Keka's integrations are rated 2.5/5, this is a place to win.

- **Dead code:** delete `CtcPlannerService`, `CostCentreService`, `DepartmentBudgetService`, `MultiPayGroupService`, `OnDemandSalaryService`, `ArrearCalculatorService`, `TaxSimulatorService`, `TaxSavingRecommender` and the endpoint that calls it. They cannot run, and they make the codebase read as more complete than it is.

11 Where you already beat Keka

Worth knowing, because these are the things to lead a competitive demo with — and things not to break during remediation.

- **ESI contribution-period lock-in.** `EsiContributionPeriodService` handles Apr–Sep / Oct–Mar correctly including the year boundary, and keeps a mid-period crosser covered by looking for an earlier contributing month. Eligibility on pre-LOP gross, contribution on post-LOP wages. Keka's docs carry a support article titled *"Why is the employee's ESI not being computed?"*
- **Form 16 Part A honesty.** Both products require a TRACES download — but CareVance sets `compliance_status = 'needs_external_input'` and explains why in code. Keka's competitive weakness here is documented; matching it and being explicit is a fair claim.
- **Bonus Act Form C/D/E generators** with the correct 8.33–20% range, ₹7,000 wage ceiling and ₹21,000 eligibility test. Keka's help documentation has no Bonus Act coverage at all. Once the engine actually pays a bonus, this is a differentiator.
- **Filing batch resilience.** `generateAllFilings` attempts each generator independently and reports failures by name — better than a first-throw-kills-the-batch design.
- **36 PT jurisdictions with a correct Maharashtra February rule.** The top-band condition is subtle and it is right.
- **UTR discipline.** The codebase states that only the bank's UTR is a reconcilable reference, and `PayrollDisbursementService` honours it. Two other endpoints violate it — fix those and this becomes a clean story.
- **Signed net pay.** Not clamping a negative net to zero, so validation can see the real number, is the correct call and rarer than it should be.
- **Tenant isolation as a structural invariant** with a test that fails on a missing trait — stronger than most multi-tenant HRMS codebases. The two payroll exceptions found here are worth closing precisely because the rest holds.
- **Queued payroll processing** that authenticates as the initiating user (so the tenant scope resolves) with `tries = 1` and a 409 on concurrent start. The reasoning is right; it needs a row lock to be airtight.

12 Verification notes

Method. Four parallel code audits against a read-only snapshot of `D:\Caretime\backend` staged 13 Aug 2026 — 206 files covering `app/{Console,Http,Jobs,Models,Services,Traits}`, `database/migrations` and `routes/`. Keka's side is from `help.keka.com` product documentation, `keka.com/payroll-software`, `academy.keka.com` and third-party review sites. `keka.com/payroll-software-features` and `/payroll-compliance` are blocked by robots.txt and could not be read.

Not covered by this audit. `resources/views/`, `tests/`, `app/Providers/` and the frontend were not staged. "Orphan" verdicts mean *no reference in the staged tree*; a container binding outside it is theoretically possible, though the services flagged as non-runnable would fatal regardless. Claims about the ten missing blade views come from `CLAUDE.md`, not from reading the view directory.

Schema drift. All schema claims are from migrations, not a live Postgres inspection. `CLAUDE.md` flags drift as a known hazard (`bank_transfer_batches`). Two findings — the missing `arrear_amount` / `rejection_reason` columns and the casing of `payroll_items.10P_days` — are worth confirming against the live database before you act on them.

Flagged as uncertain, deliberately. Current PT slab values for Karnataka, Gujarat, Odisha and Himachal Pradesh; current LWF amounts and periodicity for all 15 configured states (the code itself asks for this verification in a comment); whether `PayslipYtdHistory` is maintained; whether Keka ships payroll write APIs or webhooks. These need a domain check against gazette notifications, not a code read.

CareVance HRMS payroll audit · 13 August 2026 · every finding cites `file:line` against the staged snapshot. Re-run the audit after Phase 0 to confirm the guard consolidation actually removed the paths rather than moving them.